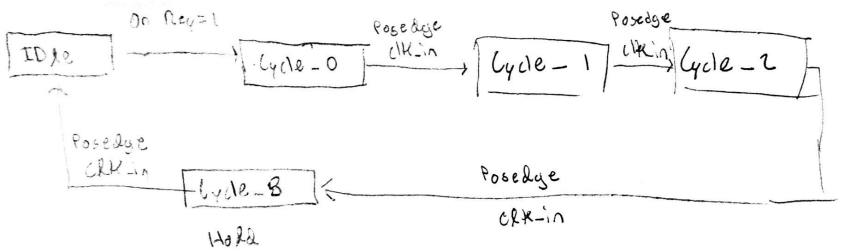
1) Memory Controller

- State Diagram:

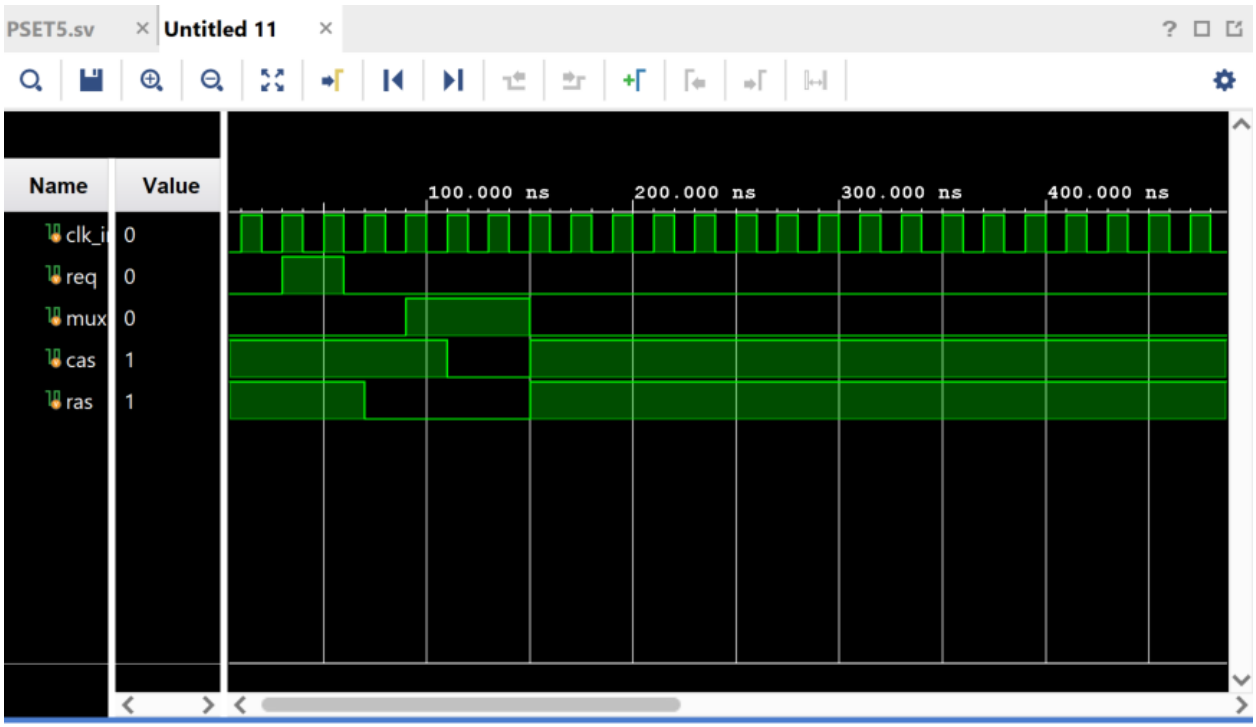


-
- Verilog:

```
module memory_controller(
    input wire clk_in,
    input wire req_in,
    output logic ras_out,
    output logic cas_out,
    output logic mux_out
    );
   logic [3:0] cycle;
   always_ff @(posedge clk_in) begin
       if(req_in) begin
          cycle <= 3'd0;
       end else begin
          case(cycle)
             3'd0 : begin ras_out <= 1'b0; cycle <= 3'd1; end
             3'd1 : begin mux_out <= 1'b1; cycle <= 3'd2; end
             3'd2 : begin cas_out <= 1'b0; cycle <= 3'd3; end
             3'd3 : begin cycle <= 3'd4; end
             default : begin mux_out <= 1'b0; cas_out <= 1'b1; ras_out <= 1'b1; end
          endcase
       end
    end
endmodule
```

- The outputs are put in registers so that when the clk pulls high it grabs the previous output values that are already stored.

- Testbench:




```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/27/2021 08:40:03 PM
// Design Name:
// Module Name: PSET5_tb
// Project Name:
// Target Devices:
// Tool Versions:
```

```
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module memory_controller_tb();
    logic clk_in;
    logic req;
    logic mux;
    logic cas;
    logic ras;

    initial req = 1'b0;
    initial mux = 1'b0;
    initial ras = 1'b1;
    initial cas = 1'b1;
    initial clk_in =1'b0;

    always begin
        #10;
        clk_in = !clk_in;
    end
   PSET5 uut(.clk_in(clk_in), .req_in(req), .ras_out(ras), .cas_out(cas), .mux_out(mux));
   initial begin
      #30;
      req = 1'b1;
      #30;
      req = 1'b0;

   end

endmodule
```

2) TMDS:

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
```

```
// Engineer:
//
// Create Date: 09/27/2021 11:37:56 PM
// Design Name:
// Module Name: stage_1
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

`default_nettype none
module stage_1 (input wire clk_in,
                input wire rst_in,
                input wire [7:0] s1_in,
                output logic [8:0] s1_out
                 );
    logic [7:0] op1;
    assign op1 = {s1_in[7] ^ s1_in[6], s1_in[6] ^ s1_in[5], s1_in[5] ^ s1_in[4], s1_in[4] ^ s1_in[3],
s1_in[3] ^ s1_in[2], s1_in[2] ^ s1_in[1], s1_in[1] ^ s1_in[0], s1_in[0]};
    logic [7:0] op2;
    assign op2 = {!(s1_in[7] ^ s1_in[6]), !(s1_in[6] ^ s1_in[5]), !(s1_in[5] ^ s1_in[4]), !(s1_in[4] ^
s1_in[3]), !(s1_in[3] ^ s1_in[2]), !(s1_in[2] ^ s1_in[1]), !(s1_in[1] ^ s1_in[0]), s1_in[0]};

    logic [3:0] tally1;
    logic [3:0] tally2;
    logic [9:0] out;
    tallier t1(.byte_in(op1), .tally_out(tally1));
    tallier t2(.byte_in(op2), .tally_out(tally2));
    assign out = (tally1 < tally2) ? {1'b1, op1} : {1'b0, op2};


   always_ff @(posedge clk_in) begin
      if(rst_in) begin
         s1_out <= 9'b0_0000_0000;
      end else begin
         s1_out <= out;
```

```
        end
    end
endmodule

module tallier(
    input wire [7:0] byte_in,
    output logic [3:0] tally_out
    );
    assign tally_out = byte_in[7] + byte_in[6] + byte_in[5] + byte_in[4] + byte_in[3] + byte_in[2] +
byte_in[1] + byte_in[0];
endmodule

`default_nettype wire
```